

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт

по лабораторной работе №2
по дисциплине «**Методы оптимизации**»

Автор: Беляева Валерия Андреевна

Факультет: ФПИиКТ

Группа: K3339

Поток: 1.3

Санкт-Петербург 2026

Лабораторная работа №2. Метод ломаных (метод Пиявского)

Задача. Найти глобальный минимум (максимум) липшицевой функции одной переменной $f(x)$ на отрезке $[a, b]$ с точностью ε по значению функции. На вход программа принимает строку с функцией, концы отрезка и ε ; на выходе — график функции, вспомогательных функций и итоговой ломаной, приближённые x^* и $f(x^*)$, число итераций и время.

Работа, как и ЛР1, выполнена в трёх видах: ручное прохождение метода, тот же расчёт в Excel (файл `excel_solution.xlsx`) и программа на Python (`python_solution.py`), которая демонстрируется на функциях Растригина и Экли.

Код программы: github.com/ValeriaBelyaeva/metopt/tree/main/lr2 (файл `python_solution.py`).

1. Метод

Функция $W(x)$ липшицева на $[a, b]$, если существует константа $L > 0$: $|W(u) - W(v)| \leq L|u - v|$ для любых $u, v \in [a, b]$. Тогда для любой точки u_n , в которой функция уже вычислена, «галочка»

$$g_n(x, u_n) = W(u_n) - L|x - u_n| \leq W(x),$$

а ломаная $p_n(x) = \max\{p_{n-1}(x), g_n(x, u_n)\} = \max_{i \leq n} g_i(x, u_i)$ — миноранта: она нигде не выше графика и с ростом числа итераций приближается к нему снизу. Её **верхние вершины** — точки $(u_i, W(u_i))$ на графике, **нижние вершины** — пересечения галочек соседних верхних вершин $u_{\text{л}} < u_{\text{п}}$:

$$u_{\text{cross}} = \frac{W(u_{\text{л}}) - W(u_{\text{п}})}{2L} + \frac{u_{\text{л}} + u_{\text{п}}}{2}, \quad p(u_{\text{cross}}) = \frac{W(u_{\text{л}}) + W(u_{\text{п}})}{2} - \frac{L(u_{\text{п}} - u_{\text{л}})}{2}.$$

Алгоритм (как в лекции): стартовые итерации $u_0 = a$, $u_1 = b$. На итерации n берём точку минимума ломаной $u_n = \arg \min p_{n-1}(x)$ — это самая низкая нижняя вершина; вычисляем $W(u_n)$ и строим $p_n = \max\{p_{n-1}, g_n\}$: нижняя вершина в u_n заменяется двумя новыми — пересечениями g_n с галочками ближайших верхних вершин слева и справа.

Остановка: $W(u_n) - p_{n-1}(u_n) < \varepsilon$; ответ $u^* = u_n$, $W^* = W(u_n)$. Поскольку $p_{n-1}(u_n) = \min p_{n-1} \leq \min W \leq W(u_n)$, погрешность по значению функции меньше ε . Максимум ищется как минимум функции — W .

Константа Липшица задаётся из оценки производной $L \geq \max |W'(x)|$, а если её нет — оценивается как на практике: $L = K \cdot \max_i \frac{|W(x_{i+1}) - W(x_i)|}{|x_{i+1} - x_i|}$, $K = 1, 2$, по вспомогательной сетке.

2. Ручное прохождение метода

Пройдём первые итерации вручную для функции из задания $W(x) = x + \sin(3,14159x)$ на $[-3; 3]$, $\varepsilon = 0,01$. Производная $W'(x) = 1 + \pi \cos(\pi x)$, поэтому $|W'(x)| \leq 1 + \pi$ и берём $L = 1 + \pi \approx 4,14159$ (оценка по формуле с практики $L = 1,2 \cdot \max |\Delta W / \Delta x|$ дала бы $\approx 4,97$ — тоже верхняя оценка, только с запасом).

Стартовые итерации. $u_0 = a = -3$: $W(u_0) = -3,00001$, галочка $g_0(x, u_0) = W(u_0) - L|x + 3|$, $p_0 = g_0$. $u_1 = b = 3$: $W(u_1) = 3,00001$, $p_1(x) = \max\{p_0(x), g_1(x, u_1)\}$.

Итерация 2. Минимум p_1 — точка пересечения галочек g_0 и g_1 :

$$u_2 = \frac{W(u_0) - W(u_1)}{2L} + \frac{u_0 + u_1}{2} = \frac{-3.00001 - 3.00001}{8,28318} + 0 = -0.72436,$$

$$p_1(u_2) = \frac{W(u_0) + W(u_1)}{2} - \frac{L(u_1 - u_0)}{2} = -12.42477.$$

$W(u_2) = -1.48607$; проверка: $W(u_2) - p_1(u_2) = 10.93870 \geq \varepsilon$ — продолжаем. Строим $g_2(x, u_2)$ и $p_2 = \max\{p_1, g_2\}$: нижняя вершина u_2 исчезает, вместо неё появляются две — пересечения g_2 с галочками ближайших верхних вершин слева (u_0) и справа (u_1):

$$[u_0, u_2] : \frac{-3.00001 - (-1.48607)}{2L} + \frac{-3 + (-0.72436)}{2} = -2.04495, \quad p = -6.95542;$$

$$[u_2, u_1] : \frac{-1.48607 - 3.00001}{2L} + \frac{-0.72436 + 3}{2} = 0.59623, \quad p = -6.95542.$$

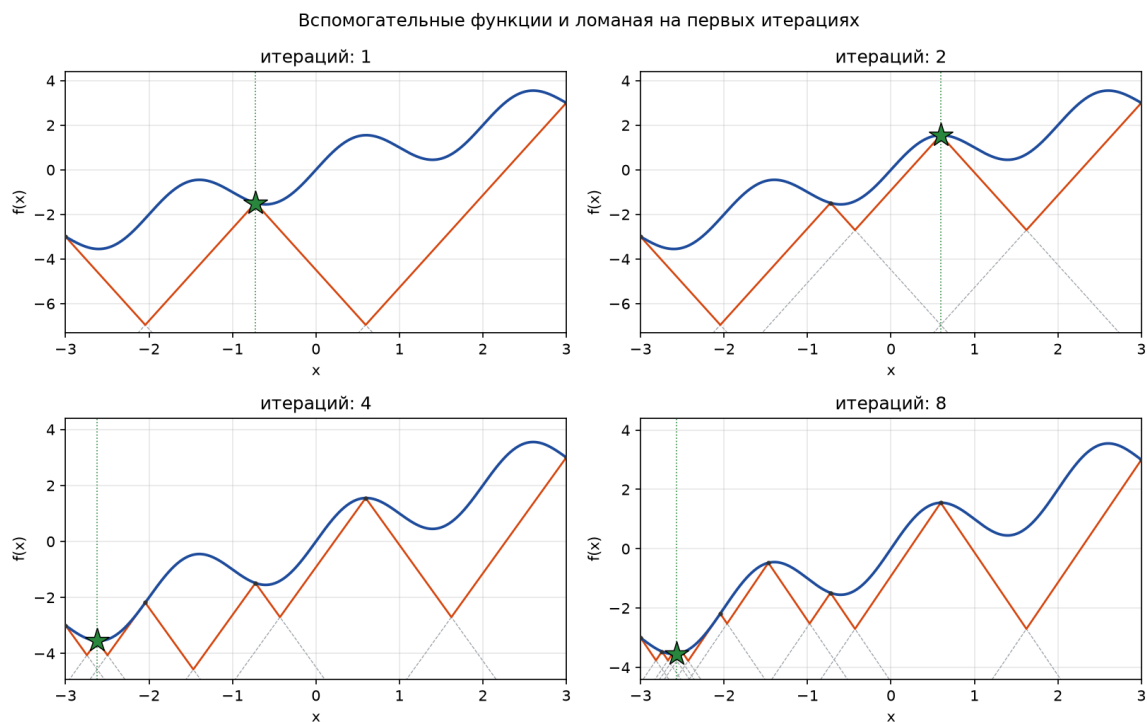
Итерация 3. Обе вершины на одной высоте -6.95542 (функция почти нечётная) — можно взять любую, программа взяла правую: $u_3 = 0.59623$, $W(u_3) = 1.55088$, разность 8.50630 — продолжаем. **Итерация 4:** $u_4 = -2.04495$, $W(u_4) = -2.18570$. **Итерация 5:** $u_5 = -2.62078$, $W(u_5) = -3.54965$ — метод уже нашёл окрестность глобального минимума и дальше уточняет её, лишь изредка проверяя другие участки, где ломаная опускается ниже.

точка	между верхними вершинами	u_n (пересечение)	$p_{n-1}(u_n)$	$W(u_n)$	$W(u_n) - p_{n-1}(u_n)$
u_2	u_0 и u_1	-0.72436	-12.42477	-1.48607	10.93870
u_3	u_2 и u_1	0.59623	-6.95542	1.55088	8.50630
u_4	u_0 и u_2	-2.04495	-6.95542	-2.18570	4.76972
u_5	u_0 и u_4	-2.62078	-4.57056	-3.54965	1.02091
u_6	u_4 и u_2	-1.46912	-4.57056	-0.47382	4.09674
u_7	u_0 и u_5	-2.74404	-4.06011	-3.46427	0.59584
u_8	u_5 и u_4	-2.49753	-4.06011	-3.49750	0.56260
u_9	u_5 и u_8	-2.56546	-3.77881	-3.54439	0.23442
...	...				
u_{51}	u_{17} и u_{37}	-2.60547	-3.55797	-3.55108	0.00689 < 0.01 — стоп

На итерации 51 (после двух стартовых — 50 итераций цикла) выполнено условие остановки $W(u_n) - p_{n-1}(u_n) < \varepsilon$:

$$u^* \approx -2.60547, \quad W(u^*) \approx -3.55108, \quad W(u^*) - p_{n-1}(u^*) = 0.00689 < 0,01.$$

Так как ломаная нигде не выше функции, $p_{n-1}(u_n) = \min p_{n-1} \leq \min W$, поэтому найденное значение отличается от глобального минимума меньше чем на ε .



Галочки g_n (пунктир) и ломаная p_n (оранжевая) после 1, 2, 4 и 8 итераций цикла

3. Расчёт в Excel

Лист «Метод ломаных» файла `excel_solution.xlsx` повторяет ручное прохождение до остановки (по образцу Excel-файла с практики: одна строка — одна галочка). Параметры a, b, L, ε — в ячейках B3:B6, $W(a), W(b)$ — B7:B8. В каждую строку вписаны верхние вершины, между которыми лежит самая низкая нижняя вершина (ссылки на ячейки уже найденных точек — это и есть «ручной» выбор шага), всё остальное считается формулами:

Столбец	Формула (строка r)	Смысл
C–F	=B\$3, =I\$12, ...	$u_{\text{л}}, W(u_{\text{л}}), u_{\text{п}}, W(u_{\text{п}})$ — ссылки на верхние вершины
G	=(Dr–Fr)/(2*B\$5)+(Cr+Er)/2	точка пересечения u_n
H	=(Dr+Fr)/2–B\$5*(Er–Cr)/2	значение ломаной $p_{n-1}(u_n)$
I	=Gr+SIN(3.14159*Gr)	$W(u_n)$ — новая верхняя вершина
J	=Ir–Hr	$W(u_n) – p_{n-1}(u_n)$
L	=IF(Jr<B\$6;"СТОП...";"продолжаем")	условие остановки

Итог на листе: 50 итераций после стартовых, $u^* \approx -2.60547$, $W(u^*) \approx -3.55108$, $W(u^*) – p(u^*) = 0.00689 < 0,01$ — совпадает с ручным расчётом и программой. На листе «График» построены $W(x)$ и все верхние вершины: видно, как они сгущаются возле глобального минимума.

4. Программа

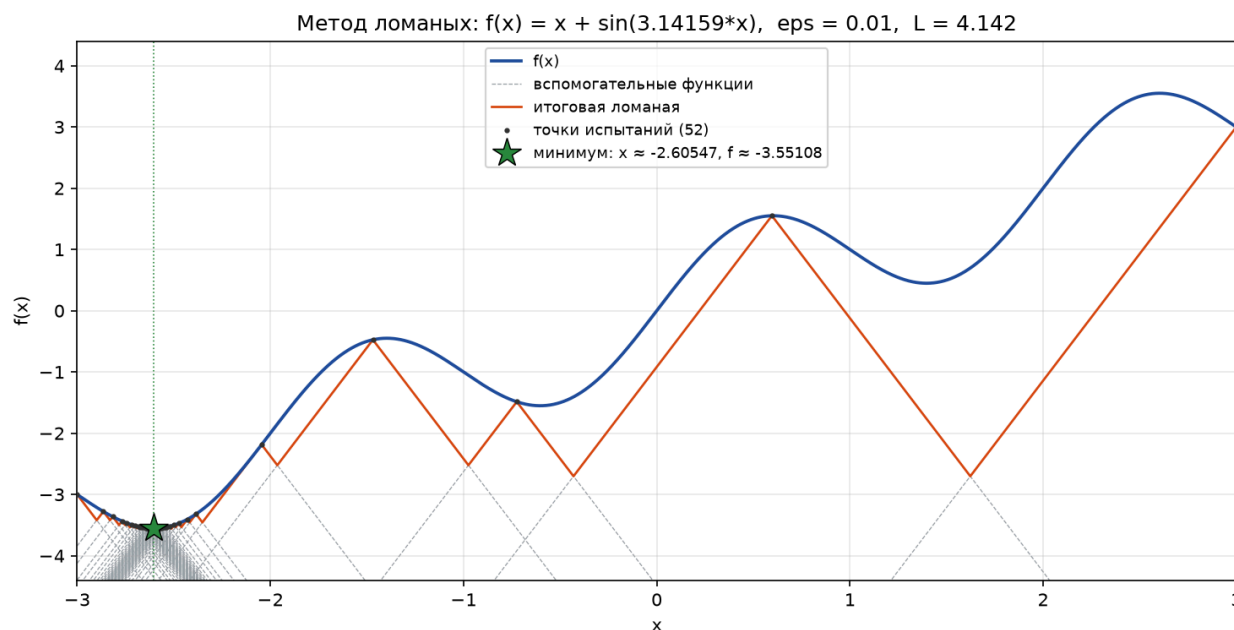
Программа `python_solution.py` (Python 3, numpy, matplotlib):

- `parse_function` — разбор строки вида $f(x) = x + \sin(3.14159 \cdot x)$ (допускаются $^$, π , e , abs , exp , тригонометрия); формула проверяется по белому списку, произвольный код не выполняется;
- `estimate_lipschitz` — оценка L по наклонам секущих с запасом 20%, если L не задана (сетка используется только для этой оценки и для рисования графика);
- `lower_vertex` — нижняя вершина ломаной на интервале (точка пересечения звеньев);
- `piyavskii` — сам метод в обозначениях лекции: верхние вершины $u_0 = a, u_1 = b, u_2, \dots$ хранятся в списке, нижние — в двоичной куче по высоте, поэтому минимум ломаной $u_n = \arg \min p_{n-1}$ находится за $O(\log n)$; остановка по условию $W(u_n) - p_{n-1}(u_n) < \varepsilon$; если точка функции оказалась ниже ломаной, программа сообщает, что L занижена;
- `plot_result, plot_steps` — график функции, вспомогательных функций, итоговой ломаной, точек испытаний и найденного экстремума;
- `solve, main` — ввод из командной строки или с клавиатуры, печать результатов, сохранение графика.

Запуск: `python3 python_solution.py "x + sin(3.14159*x)" -3 3 0,01` (ключи `--L`, `--max`), без аргументов — ввод с клавиатуры, `--demo` — все примеры ниже.

4.1. Пример из задания: $f(x) = x + \sin(3,14159x)$ на $[-3, 3]$, $L = 1 + \pi$

```
Функция:      f(x) = x + sin(3.14159*x)
Отрезок:      [-3, 3],  eps = 0.01,  ищем минимум
Константа Липшица: L = 4.14159  (задана)
Аргумент:     x* ≈ -2.60546973
Значение функции: f(x*) ≈ -3.55107830
Ломаная в u*:  p(u*) = -3.55796559  (W(u*) - p(u*) = 6.887e-03 < eps: True)
Число итераций: 50  (+ 2 стартовые, вычислений функции: 52)
Затраченное время: 0.063 мс
График:       figures/1_example.png
```

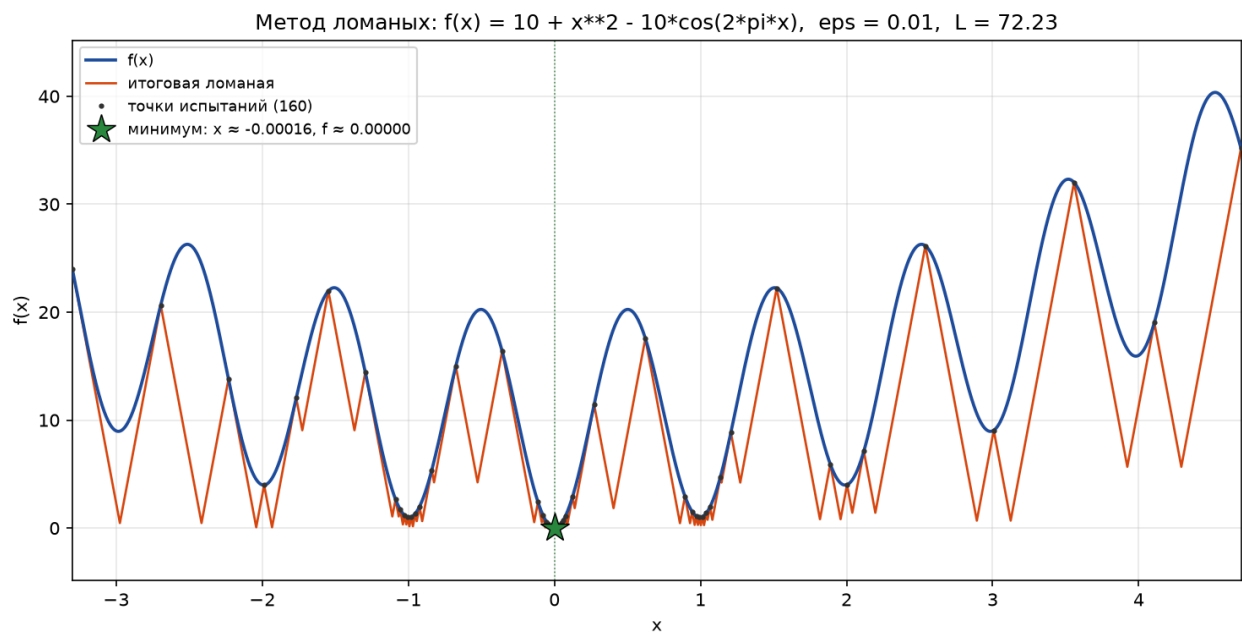


Метод ломаных для функции из задания: три локальных минимума, найден глобальный

4.2. Функция Растригина (одномерный срез)

$f(x) = 10 + x^2 - 10 \cos(2\pi x)$ на $[-3,3; 4,7]$ — восемь локальных минимумов, глобальный $f(0) = 0$ (отрезок несимметричный, чтобы метод не попал в $x = 0$ случайно). $|f'(x)| = |2x + 20\pi \sin(2\pi x)| \leq 9,4 + 20\pi$, поэтому $L = 9,4 + 20\pi \approx 72,23$.

```
Функция:      f(x) = 10 + x**2 - 10*cos(2*pi*x)
Отрезок:      [-3.3, 4.7], eps = 0.01, ищем минимум
Константа Липшица: L = 72.2319 (задана)
Аргумент:     x* ≈ -0.00015688
Значение функции: f(x*) ≈ 0.00000488
Ломаная в u*:  p(u*) = -0.00688656 (W(u*) - p(u*) = 6.891e-03 < eps: True)
Число итераций: 158 (+ 2 стартовые, вычислений функции: 160)
Затраченное время: 0.173 мс
График:       figures/2_rastrigin.png
```

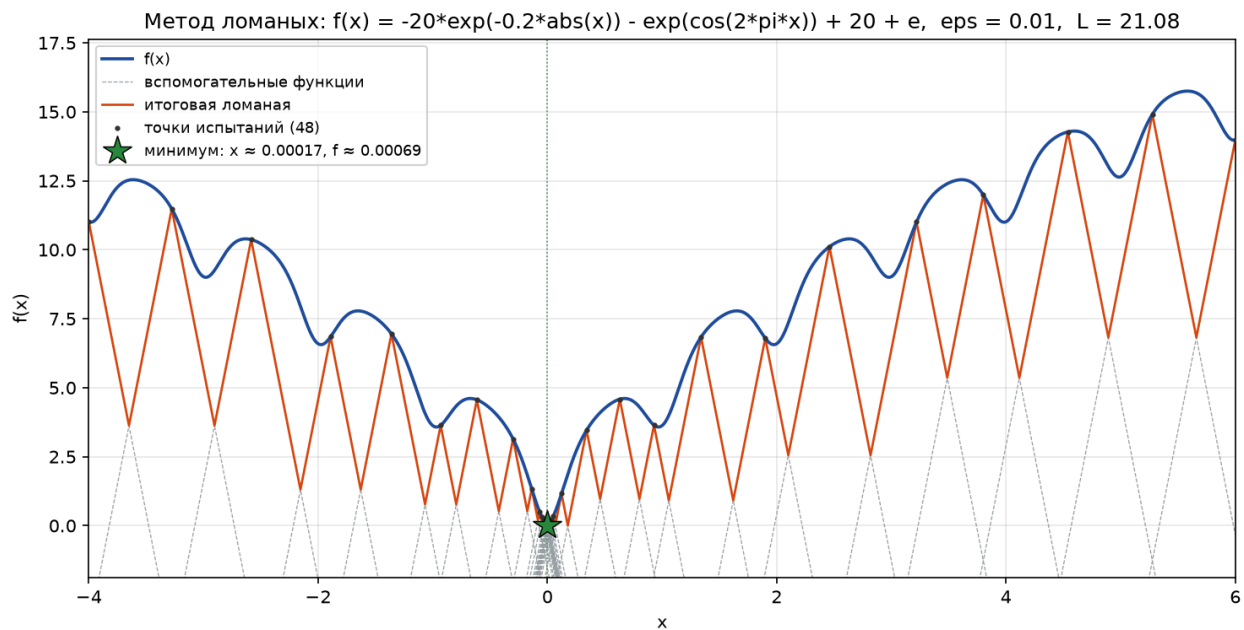


Функция Растригина: испытания сгущаются у глобального и у «конкурирующих» локальных минимумов

4.3. Функция Экли (одномерный срез)

$f(x) = -20e^{-0,2|x|} - e^{\cos 2\pi x} + 20 + e$ на $[-4, 6]$, глобальный минимум $f(0) = 0$; в точке минимума функция негладкая, но липшицева: $L = 4 + 2\pi e \approx 21,08$.

```
Функция:      f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e
Отрезок:      [-4, 6], eps = 0.01, ищем минимум
Константа Липшица: L = 21.0795 (задана)
Аргумент:     x* ≈ 0.00017182
Значение функции: f(x*) ≈ 0.00068886
Ломаная в u*:  p(u*) = -0.00419322 (W(u*) - p(u*) = 4.882e-03 < eps: True)
Число итераций: 46 (+ 2 стартовые, вычислений функции: 48)
Затраченное время: 0.064 мс
График:       figures/3_ackley.png
```



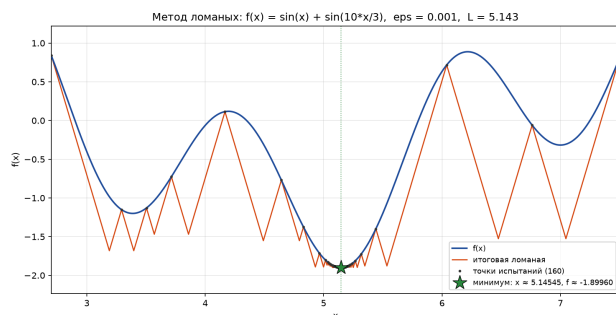
Функция Экли

4.4. Автоматическая оценка L , поиск минимума и максимума

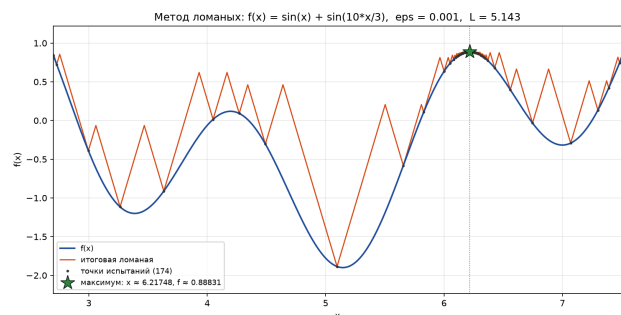
$f(x) = \sin x + \sin \frac{10x}{3}$ на $[2,7; 7,5]$, $\varepsilon = 0,001$; L не задаётся.

Функция: $f(x) = \sin(x) + \sin(10 \cdot x/3)$
 Отрезок: $[2.7, 7.5]$, $\text{eps} = 0.001$, ищем минимум
 Константа Липшица: $L = 5.14277$ (оценка по сетке с запасом 20%)
 Аргумент: $x^* \approx 5.14545450$
 Значение функции: $f(x^*) \approx -1.89959888$
 Ломаная в u^* : $p(u^*) = -1.90044304$ ($W(u^*) - p(u^*) = 8.442e-04 < \text{eps}$: True)
 Число итераций: 158 (+ 2 стартовые, вычислений функции: 160)
 Затраченное время: 0.250 мс
 График: figures/4_min.png

Функция: $f(x) = \sin(x) + \sin(10 \cdot x/3)$
 Отрезок: $[2.7, 7.5]$, $\text{eps} = 0.001$, ищем максимум
 Константа Липшица: $L = 5.14277$ (оценка по сетке с запасом 20%)
 Аргумент: $x^* \approx 6.21747901$
 Значение функции: $f(x^*) \approx 0.88831463$
 Ломаная в u^* : $p(u^*) = 0.88922721$ ($W(u^*) - p(u^*) = 9.126e-04 < \text{eps}$: True)
 Число итераций: 172 (+ 2 стартовые, вычислений функции: 174)
 Затраченное время: 0.247 мс
 График: figures/4_max.png



минимум



максимум (ломаная сверху)

4.5. Проверка и влияние точности

Для Растригина и Экли точный ответ известен ($x^* = 0$, $f^* = 0$): фактическая погрешность по значению $4,9 \cdot 10^{-6}$ и $6,9 \cdot 10^{-4}$ — меньше $\varepsilon = 0,01$. Зависимость числа итераций от ε

(Растригин); для сравнения — сколько точек равномерной сетки дало бы ту же гарантию: $N = L(b - a)/(2\varepsilon)$:

eps	итераций	время, мс	x*	f(x*)	сетка с той же гарантией
0.1	78	0.091	-0.0021743	0.0009379	2890
0.01	158	0.164	-0.0001569	0.0000049	28893
0.001	398	0.392	-0.0000019	0.0000000	288928
0.0001	1508	1.555	-0.0000012	0.0000000	2889275
1e-05	4194	4.370	+0.0000002	0.0000000	28892742

Листинг python_solution.py

```

"""
ЛР2. Метод ломаных (метод Пиявского) – глобальный экстремум липшицевой функции на отрезке.
Беляева Валерия Андреевна, КЗ339, поток 1.3.

Запуск:
python3 python_solution.py          – ввод f(x), a, b, eps (и L) с клавиатуры
python3 python_solution.py "x + sin(3.14159*x)" -3 3 0.01 [--L 4.1416] [--max]
python3 python_solution.py --demo   – все демонстрации из отчёта (графики в figures/)

Экстремум ищется только методом ломаных: новые точки испытаний – аналитически вычисленные
нижние вершины ломаной (точки пересечения её звеньев), верхние вершины – точки испытаний.
Равномерная сетка используется лишь для рисования графика и (если L не задана) для оценки L.
"""
import argparse
import ast
import heapq
import time
from pathlib import Path

import numpy as np
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

FIG_DIR = Path(__file__).resolve().parent / "figures"

# ----- разбор строки f(x) -----

_NAMES = {name: getattr(np, name) for name in
["sin", "cos", "tan", "exp", "sqrt", "log", "log2", "log10",
"arcsin", "arccos", "arctan", "sinh", "cosh", "tanh", "floor", "ceil", "sign"]}
_NAMES.update({"abs": np.abs, "ln": np.log, "asin": np.arcsin, "acos": np.arccos,
"atan": np.arctan, "pi": np.pi, "e": np.e})
_NODES = (ast.Expression, ast.BinOp, ast.UnaryOp, ast.Call, ast.Name, ast.Load, ast.Constant,
ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow, ast.Mod, ast.USub, ast.UAdd)

def parse_function(text):
    """Строка 'f(x) = x + sin(3.14159*x)' -> функция f(x).
    Левую часть 'f(x) =' можно не писать, '^' понимается как степень.
    Формула проверяется по белому списку (никакого произвольного eval)."""
    expr = text.strip()
    if "=" in expr:
        expr = expr.split("=", 1)[1]
    expr = expr.strip().replace("^", "**")
    tree = ast.parse(expr, mode="eval")
    for node in ast.walk(tree):
        if not isinstance(node, _NODES):
            raise ValueError(f"Недопустимая конструкция в формуле: {type(node).__name__}")
        if isinstance(node, ast.Name) and node.id != "x" and node.id not in _NAMES:
            raise ValueError(f"Неизвестное имя в формуле: {node.id}")
        if isinstance(node, ast.Call) and not isinstance(node.func, ast.Name):
            raise ValueError(f"Недопустимый вызов в формуле")
    code = compile(tree, "<f(x)>", "eval")

    def f(x):
        return eval(code, {"__builtins__": {}}, {_NAMES, "x": x})

    f.expr = expr
    return f

```

```

def to_float(value):
    """Число или строка с числом; допускается десятичная запятая ('0,01')."""
    if isinstance(value, str):
        value = value.strip().replace(",", ".")
    return float(value)

# ----- константа Липшица

def estimate_lipschitz(f, a, b, n=2000, reserve=1.2):
    """Оценка L, если она не задана:  $\max |\Delta f / \Delta x|$  по вспомогательной сетке с запасом 20%. """
    xs = np.linspace(a, b, n + 1)
    ys = np.asarray(f(xs), dtype=float) * np.ones_like(xs)
    slope = np.max(np.abs(np.diff(ys) / np.diff(xs)))
    return max(reserve * slope, 1e-12)

# ----- метод ломаных

def lower_vertex(xl, fl, xr, fr, L):
    """Нижняя вершина ломаной на [xl, xr] – пересечение прямых
     $y = fl - L(x - xl)$  и  $y = fr + L(x - xr)$ . """
    xv = 0.5 * (xl + xr) + (fl - fr) / (2.0 * L)
    pv = 0.5 * (fl + fr) - 0.5 * L * (xr - xl)
    return pv, xv

def piyavskii(f, a, b, L, eps, mode="min", max_iter=1_000_000):
    """Глобальный экстремум липшицевой  $W(x) = f(x)$  на [a, b] методом ломаных (обозначения лекции).

     $u_0 = a, u_1 = b$  – стартовые итерации;  $g_n(x, u_n) = W(u_n) - L|x - u_n|$  – «галочка»;
     $p_n(x) = \max\{p_{n-1}(x), g_n(x, u_n)\}$  – ломаная (миноранта).
    Итерация n:  $u_n = \operatorname{argmin} p_{n-1}(x)$  – самая низкая нижняя вершина (точка пересечения
    соседних галочек); остановка, когда  $W(u_n) - p_{n-1}(u_n) < \text{eps}$ ; ответ  $u^* = u_n, W^* = W(u_n)$ .
    mode = 'min' или 'max' (максимум W = минимум -W).
    Возвращает словарь с ответом, историей итераций и данными для графика. """
    if not a < b:
        raise ValueError("Должно быть a < b")
    if L <= 0 or eps <= 0:
        raise ValueError("L и eps должны быть положительными")
    sign = 1.0 if mode == "min" else -1.0
    g = lambda x: sign * float(f(x))

    t0 = time.perf_counter()
    ga, gb = g(a), g(b)
    if abs(ga - gb) > L * (b - a):
        raise ValueError("Константа L занижена: условие Липшица нарушено на концах отрезка")
    trial_x, trial_g = [a, b], [ga, gb]  # верхние вершины:  $u_0 = a, u_1 = b, u_2, \dots$ 

    # куча нижних вершин: (высота p, абсцисса, индекс левой верхней вершины, индекс правой)
    pv, xv = lower_vertex(a, ga, b, gb, L)
    heap = [(pv, xv, 0, 1)]
    history = []
    iterations = 0
    while True:
        pv, xv, il, ir = heapq.heappop(heap)  #  $u_n$  – минимум ломаной  $p_{n-1}$ ,  $p_v = p_{n-1}(u_n)$ 
        iterations += 1
        gv = g(xv)  #  $W(u_n)$  – новое испытание
        if gv < pv - 1e-9 * max(1.0, abs(pv)):
            raise ValueError(f"Константа L занижена: f({xv:.6g}) лежит ниже ломаной")
        delta = gv - pv  #  $W(u_n) - p_{n-1}(u_n)$ 
        stop = delta < eps or iterations >= max_iter
        history.append({"k": iterations, "il": il, "ir": ir, "xv": xv, "pv": pv, "gv": gv,
                       "delta": delta, "stop": stop})
        trial_x.append(xv)
        trial_g.append(gv)
        new = len(trial_x) - 1
        # галочка  $g_n$  заменяет вершину двумя новыми – слева и справа от  $u_n$ 
        pl, xvl = lower_vertex(trial_x[il], trial_g[il], xv, gv, L)
        pr, xvr = lower_vertex(xv, gv, trial_x[ir], trial_g[ir], L)
        heapq.heappush(heap, (pl, xvl, il, new))
        heapq.heappush(heap, (pr, xvr, new, ir))
        if stop:
            break
    elapsed = time.perf_counter() - t0

    return {
        "mode": mode, "a": a, "b": b, "L": L, "eps": eps, "sign": sign,
        "x": xv, "f": sign * gv,  # ответ по лекции:  $u^* = u_n, W^* = W(u_n)$ 
        "bound": sign * pv,  #  $p_{n-1}(u_n)$  – минимум ломаной, оценка снизу
    }

```

```

        "delta": delta,                                     #  $W(u_n) - p_{n-1}(u_n)$ 
        "iterations": iterations,                          # итерации после стартовых ( $u_0 = a, u_1 = b$ )
        "evaluations": len(trial_x),
        "time": elapsed,
        "converged": delta < eps,
        "trial_x": np.array(trial_x),
        "trial_f": sign * np.array(trial_g),
        "vert_x": np.array([h[1] for h in heap]),           # нижние вершины итоговой ломаной  $p_n$ 
        "vert_p": sign * np.array([h[0] for h in heap]),
        "history": history,
    }

# ----- визуализация

def broken_line(res):
    """Итоговая ломаная: чередование точек испытаний и нижних вершин."""
    x = np.concatenate([res["trial_x"], res["vert_x"]])
    y = np.concatenate([res["trial_f"], res["vert_p"]])
    order = np.argsort(x, kind="stable")
    return x[order], y[order]

def plot_result(f, res, title="", ax=None, max_aux=60, legend=True):
    a, b, L, sign = res["a"], res["b"], res["L"], res["sign"]
    own = ax is None
    if own:
        fig, ax = plt.subplots(figsize=(10, 5.2))
        xx = np.linspace(a, b, 2000)                       # сетка только для рисования
        yy = np.asarray(f(xx), dtype=float) * np.ones_like(xx)
        ax.plot(xx, yy, color="#1f4e9c", lw=1.8, label="f(x)", zorder=3)
        if len(res["trial_x"]) <= max_aux:                 # вспомогательные функции  $f(x_i) \mp L|x - x_i|$ 
            for i, (xi, fi) in enumerate(zip(res["trial_x"], res["trial_f"])):
                ax.plot(xx, fi - sign * L * np.abs(xx - xi), color="#9aa0a6", lw=0.6, ls="--",
                        label="вспомогательные функции" if i == 0 else None, zorder=1)
        bx, by = broken_line(res)
        ax.plot(bx, by, color="#d9480f", lw=1.3, label="итоговая ломаная", zorder=2)
        ax.plot(res["trial_x"], res["trial_f"], ".", color="#333333", ms=4,
                label=f"точки испытаний ({res['evaluations']})", zorder=4)
        kind = "минимум" if res["mode"] == "min" else "максимум"
        ax.plot([res["x"]], [res["f"]], "*", color="#2b8a3e", ms=17, mec="black", mew=0.7, zorder=5,
                label=f"{kind}:  $x \approx \{res['x']:.5f\}, f \approx \{res['f']:.5f\}$ ")
        ax.axvline(res["x"], color="#2b8a3e", lw=0.8, ls=":", zorder=1)
        span = max(yy.max() - yy.min(), 1e-9)
        lo, hi = yy.min() - 0.12 * span, yy.max() + 0.12 * span
        if res["mode"] == "min":
            lo = min(lo, max(by.min(), yy.min() - 0.6 * span) - 0.05 * span)
        else:
            hi = max(hi, min(by.max(), yy.max() + 0.6 * span) + 0.05 * span)
        ax.set_ylim(lo, hi)
        ax.set_xlim(a, b)
        ax.set_xlabel("x")
        ax.set_ylabel("f(x)")
        ax.set_title(title)
        ax.grid(alpha=0.3)
        if legend:
            ax.legend(loc="best", fontsize=8.5)
        if own:
            fig.tight_layout()
            return fig

def plot_steps(f, a, b, L, steps=(1, 2, 4, 8), mode="min"):
    """Как уточняется ломаная на первых итерациях."""
    fig, axes = plt.subplots(2, 2, figsize=(11, 7))
    for ax, k in zip(axes.ravel(), steps):
        res = piyavskii(f, a, b, L, eps=1e-12, mode=mode, max_iter=k)
        plot_result(f, res, title=f"итераций: {k}", ax=ax, legend=False)
    fig.suptitle("Вспомогательные функции и ломаная на первых итерациях")
    fig.tight_layout()
    return fig

# ----- запуск

def solve(func_str, a, b, eps, L=None, mode="min", fig_name=None):
    """Разбор функции, метод ломаных, печать результатов, график."""
    f = parse_function(func_str)
    a, b, eps = to_float(a), to_float(b), to_float(eps)
    auto = L is None

```

```

L = estimate_lipschitz(f, a, b) if auto else to_float(L)
res = piyavskii(f, a, b, L, eps, mode=mode)

kind = "минимум" if mode == "min" else "максимум"
print(f"Функция:      f(x) = {f.expr}")
print(f"Отрезок:      [{a:g}, {b:g}],  eps = {eps:g},  ищем {kind}")
print(f"Константа Липшица: L = {L:.6g}" + (" (оценка по сетке с запасом 20%)" if auto else "
(задана)"))
print(f"Аргумент:      x* ≈ {res['x']:.8f}")
print(f"Значение функции: f(x*) ≈ {res['f']:.8f}")
print(f"Ломаная в уз:      p(u*) = {res['bound']:.8f}    (W(u*) - p(u*) = {res['delta']:.3e} < eps:
{res['converged']})")
print(f"Число итераций:      {res['iterations']}    (+ 2 стартовые, вычислений функции:
{res['evaluations']})")
print(f"Затраченное время: {res['time'] * 1000:.3f} мс")

fig = plot_result(f, res, title=f"Метод ломаных: f(x) = {f.expr},  eps = {eps:g},  L = {L:.4g}")
if fig_name:
    FIG_DIR.mkdir(exist_ok=True)
    fig.savefig(FIG_DIR / fig_name, dpi=150)
    print(f"График:      figures/{fig_name}")
plt.close(fig)
print()
return res

def demo():
    """Демонстрации из отчёта."""
    FIG_DIR.mkdir(exist_ok=True)
    print("=== 1. Пример из задания: f(x) = x + sin(3.14159x) на [-3, 3] ===")
    solve(f"x + sin(3.14159*x)", -3, 3, "0.01", L=1 + 3.14159, fig_name="1_example.png")
    fig = plot_steps(parse_function("x + sin(3.14159*x)"), -3, 3, 1 + 3.14159)
    fig.savefig(FIG_DIR / "1_steps.png", dpi=150)
    plt.close(fig)

    print("=== 2. Функция Растригина: 10 + x^2 - 10cos(2πx) на [-3.3, 4.7] ===")
    L_rastrigin = 2 * 4.7 + 20 * np.pi
    solve("10 + x^2 - 10*cos(2*pi*x)", -3.3, 4.7, 0.01, L=L_rastrigin, fig_name="2_rastrigin.png")

    print("=== 3. Функция Экли: -20exp(-0.2|x|) - exp(cos 2πx) + 20 + e на [-4, 6] ===")
    L_ackley = 4 + 2 * np.pi * np.e
    solve("-20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e", -4, 6, 0.01, L=L_ackley,
        fig_name="3_ackley.png")

    print("=== 4. sin(x) + sin(10x/3) на [2.7, 7.5]: L оценивается автоматически, min и max ===")
    solve("sin(x) + sin(10*x/3)", 2.7, 7.5, 0.001, fig_name="4_min.png")
    solve("sin(x) + sin(10*x/3)", 2.7, 7.5, 0.001, mode="max", fig_name="4_max.png")

    print("=== 5. Влияние eps (Растригин) ===")
    f_r = parse_function("10 + x^2 - 10*cos(2*pi*x)")
    print(f"{'eps':>8} | {'итераций':>9} | {'время, мс':>10} | {'x*':>11} | {'f(x*)':>11} | {'сетка с
той же гарантией':>24}")
    print("-" * 88)
    for eps in (1e-1, 1e-2, 1e-3, 1e-4, 1e-5):
        r = piyavskii(f_r, -3.3, 4.7, L_rastrigin, eps)
        n_grid = int(np.ceil(L_rastrigin * 8.0 / (2 * eps)))
        print(f"{'eps':8g} | {'iterations':9d} | {'time': * 1000:10.3f} | {'x':+11.7f} | "
            f"{'f':+11.7f} | {'n_grid':24d}")

def main():
    p = argparse.ArgumentParser(description="Метод ломаных (Пиявского)")
    p.add_argument("func", nargs="?", help="функция, например \"x + sin(3.14159*x)\"")
    p.add_argument("a", nargs="?", help="a")
    p.add_argument("b", nargs="?", help="b")
    p.add_argument("eps", nargs="?", help="eps")
    p.add_argument("--L", help="константа Липшица (если не задана – оценивается)")
    p.add_argument("--max", action="store_true", help="искать максимум вместо минимума")
    p.add_argument("--demo", action="store_true", help="запустить демонстрации из отчёта")
    args = p.parse_args()

    if args.demo:
        demo()
        return
    if args.func is None:
        # интерактивный ввод
        args.func = input("f(x) = ")
        args.a = input("a = ")
        args.b = input("b = ")
        args.eps = input("eps = ")
        args.L = input("L (Enter – оценить автоматически) = ") or None

```

```

solve(args.func, args.a, args.b, args.eps, L=args.L, mode="max" if args.max else "min",
      fig_name="result.png")

if __name__ == "__main__":
    main()

```

5. Выводы

1. Реализован метод ломаных для поиска глобального минимума и максимума липшицевой функции одной переменной; функция задаётся строкой, результат — u^* , $W(u^*)$, достигнутая точность, число итераций, время и график.
2. Новые точки u_n вычисляются аналитически как точки пересечения галочек; нижние вершины хранятся в куче, верхние — в списке точек испытаний. Перебора отрезка по мелкой сетке для поиска экстремума нет.
3. Остановка — по критерию из лекции $W(u_n) - p_{n-1}(u_n) < \varepsilon$, который гарантирует, что найденное значение отличается от глобального минимума меньше чем на ε .
4. Ручной расчёт, Excel и программа на функции из задания дают одинаковые 50 итераций и $u^* \approx -2.60547$, $W(u^*) \approx -3.55108$.
5. На функциях Растригина и Экли метод находит именно глобальный минимум среди нескольких локальных, с погрешностью меньше ε , и тратит на порядки меньше вычислений функции, чем равномерная сетка с той же гарантией.

Рефлексивный вывод

Самым наглядным оказалось ручное прохождение и таблица в Excel: после нескольких шагов видно, что метод не тратит испытания на участки, где ломаная уже выше найденных значений, и сам «сбегается» к глобальному минимуму. Главная тонкость — константа Липшица: если её понизить, метод теряет гарантию и может пропустить минимум (поэтому в программе есть проверка, что точка функции не оказалась ниже ломаной), а если сильно завysить — зубцы становятся острее и итераций нужно больше. Пришлось разобраться, как получить L из оценки производной и как аккуратно хранить вершины, чтобы не перебирать отрезок, — за перебор по сетке в задании снимают баллы, и это хорошо показывает, в чём вообще идея метода.